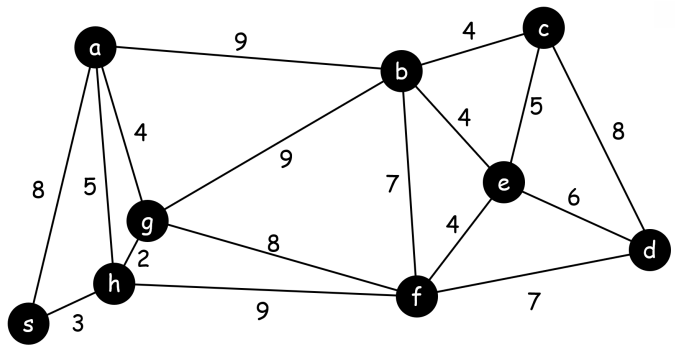
CISC2006: Algorithm Design and Analysis
Homework 4 **Due Date:** 23:59, 25 April 2025

Rules: Discussion of the problems is permitted, but writing the assignment together is not (i.e. you are not allowed to see the actual pages of another student). You can get at most 100 points if attempting all problems. Please make your answers precise and concise.

1. (10 pts) Consider the problem of computing the Minimum Spanning Tree (MST) of the following edge-weighted graph.



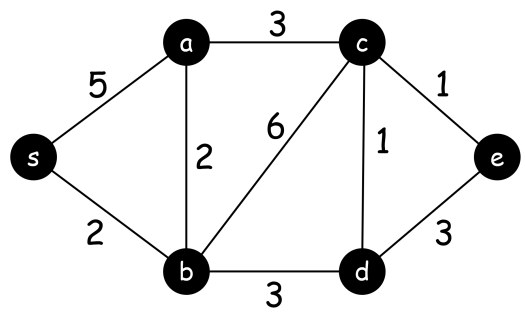
- List the edges that appear in the MST in the order they are included using Kruskal's algorithm. For example, if edge (a, b) is included as the first edge in the MST and (c, d) is the second,..., then the list is $(a, b), (c, d), \ldots$.

  Solution:
  (g, h)
  (s, h)
  $(a, g), (b, c), (b, e), (e, f)$
  (e, d)
  (g, f)

- List the edges that appear in the MST in the order they are included using Prim's algorithm. Suppose we begin with node s.

  Solution:
  (s, h)
  (h, g)
  (a, g)
  (g, f)
  (f, e)
  (e, b)
  (b, c)
  (e, d)

2. (15 pts) Consider running Dijkstra's Algorithm on the following graph to compute the distance $d(s, u)$ between s and every other node $u \in V$.



Let $S \subseteq V$ contain the set of nodes u whose distance $d(s, u)$ is known.

Initialization: set $d'(s, a) =$ __5__, $d'(s, b) = 2$, $d'(s, c) = d'(s, d) = d'(s, e) = \infty$.

Pick the node b with the smallest estimate, and include it into S: $S \leftarrow S \cup \{b\}$; set $d(s, b) = 2$.

Update the estimates of nodes in $V \setminus S$ as follows:

$d'(s, a) = \min\{5, d(s, b) +$ __w_{ab}__$\} =$ __4__, $d'(s, c) =$ __8__

$d'(s, d) =$ __5__, $d'(s, e)$ remains the same value as before.

Pick the node a with the smallest estimate, and include it into S: $S \leftarrow S \cup \{a\}$; set $d(s, a) =$ __4__.

Update the estimates of nodes in $V \setminus S$ as follows:

$d'(s, c) = \min\{8, d(s, a) +$ __w_{ac}__$\} =$ __7__.

$d'(s, d)$ and $d'(s, e)$ remain the same values as before.

Pick the node __d__ with the smallest estimate, and include it into S: __$S \leftarrow S \cup \{d\}$__; set $d(s,$ __d__$) =$ __5__.

Update the estimates of nodes in $V \backslash S$ as follows: __$d'(s, c) = \min\{7, d(s, d) + w_{cd}\} = 6$__, __$d'(s, e) = d(s, d) + w_{de} = 8$__.

Pick the node __c__ with the smallest estimate, and include it into S: __$S \leftarrow S \cup \{c\}$__; set $d(s,$ __c__$) =$ __6__.

Update the estimates of nodes in $V \backslash S$ as follows: $d'(s, e) =$ __$\min\{8, d(s, c) + w_{ce}\} = 7$__.

Pick the node e with the smallest estimate, and include it into S: $S \leftarrow S \cup \{e\}$; set $d(s, e) = 7$. Since $S = V$, the algorithm terminates.

3. (15 pts) Recall that the degree $d(u)$ of a node u in a graph is the number of neighbors of u. Prove the following statements.

- (5pts) In any graph, the number of nodes with odd degree must be even.

  Solution: Recall that given any graph with n nodes and m edges we have $\sum_{u \in V} d(u) = 2m$. Therefore the sum of degrees of all nodes must be an even numbers, which implies that the number of nodes with odd degree must be even.

- (5pts) Every tree with $n \geq 2$ nodes contains at least two nodes with degree 1.

  Solution: Recall that given any graph with n nodes and m edges we have $\sum_{u \in V} d(u) = 2m$. Also recall that every tree is a connected graph with $m = n-1$. Since the graph is connected we have $d(u) \geq 1$ for all $u \in V$.

  Next we prove the statement by contradiction.

  Suppose that there is at most one node with degree 1. Then we have at least $n-1$ nodes with degree ≥ 2, which implies

  $$\sum_{u \in V} d(u) \geq (n-1) \cdot 2 + 1 = 2n - 1 > 2 \cdot m,$$

  which is a contradiction.

- (5pts) If every node of a graph G has degree at least $n/2$, where n is the total number of nodes in the graph, then G is connected.

  Solution: We prove by contradiction and assume that G contains two or more connected components. Let $V' \subsetneq V$ be the set of nodes in the smallest connected component. Then we have $|V'| \leq n/2$. Therefore every node in V' has degree at most $|V'| - 1 \leq n/2 - 1$ in G, which is a contradiction.

4. (20 pts) Prove the following statements.

- (10 pts) Any complete graph K_n with $n \geq 3$ is not bipartite.

  Solution: Any complete graph K_n with $n \geq 3$ contains three nodes that are pairwise connected. Hence it contains a cycle of length 3. By the property of bipartite graph, any graph that contains an odd cycle is not bipartite.

- (10 pts) Any graph $G(V, E)$ with n nodes, if there exists an edge $(u, v) \in E$ such that $d(u) + d(v) > n$, then the graph is not bipartite.

  Solution: Assume for contradiction that the graph is bipartite, where V is partitioned into two sets L and R. Since there is an edge between nodes u and v, the two nodes cannot be on the same side of the graph. Note that the degree of a node on one side is upper bounded by the number of nodes on the other side. Therefore we have $d(u) + d(v) \leq |L| + |R| = n$, which is a contradiction.

5. (20 pts) **Implementation of the Graph Traversal Algorithms**.

In this problem you need to implement the graph traversal algorithms using DFS and BFS. In the main function, your algorithm needs to (1) read an integer n that denotes the number of nodes; (2) read a sequence of edges, where each edge is specified by (i, j), indicating that there is an edge between node i and node j.

You are guaranteed that both $i, j \in \{1, 2, \ldots, n\}$.

Based on the input, you need to construct an undirected unweighted graph using adjacency lists. That is, every node u stores its set of incident edges as a list.

Then a function is called to traverse the graph from node 1, and output the sequence of nodes in the order they are visited by the algorithm.

You need to implement each of the following 2 algorithms as a function.

- Depth First Search (DFS);
- Breadth First Search (BFS).

Several test cases of input will be provided. For each test case, the main function calls the two traversal algorithms, each of them outputs a traversal sequence of nodes.

6. (20 pts) **Implementing the Dijkstra's Algorithm**.

In this problem you need to implement the Dijkstra's algorithms for computing the single source shortest path (SSSP) problem. In the main function, your algorithm needs to (1) read an integer n that denotes the number of nodes; (2) read a sequence of edges, where each edge is specified by (i, j, w), indicating that there is an edge between node i and node j with integer edge weight w.

You are guaranteed that both $i, j \in \{1, 2, \ldots, n\}$ and $w \geq 1$.

Based on the input, you need to construct an undirected weighted graph using adjacency lists. That is, every node u stores its set of incident edges as a list. Then a function is called to compute the distance $d(1, i)$ between node 1 and node i, for every $i \in \{2, 3, \ldots, n\}$, and output the distances $d(1, 2), d(1, 3), \ldots, d(1, n)$.

Recall that to implement Dijkstra's algorithm we need to update and maintain an estimate $d'(1, i)$ for the distance between 1 and i, for every $i \in \{2, 3, \ldots, n\}$. You need to store the estimates in a Binary Search Tree (BST) that supports the query of minimum value. You can import libraries for the BST data structure.

Several test cases of input will be provided. For each test case, the main function calls the Dijkstra's algorithm to compute and output the distances.

7. (20 pts) **Implementation of Algorithms for the MST Problem**.

In this problem you need to implement the Kruskal's algorithm and Prim's algorithm for the Minimum Spanning Tree (MST) problem. In the main function, your algorithm needs to (1) read an integer n that denotes the number of nodes; (2) read a sequence of edges, where each edge is specified by (i, j, w), indicating that there is an edge between node i and node j with integer edge weight w.

You are guaranteed that $i, j \in \{1, 2, \ldots, n\}$ and $w \geq 1$.

Based on the input, you need to construct an undirected weighted graph using adjacency lists. That is, every node u stores its set of incident edges as a list. Then a function is called to compute the MST of the input graph, output the edges in the MST and also output the total edge weight in the MST.

You need to implement each of the following 2 algorithms as a function.

- Kruskal's algorithm that repeatedly includes an edge with minimum weight as long as it does not create a cycle. Recall that to test whether a new edge creates a cycle or not, you need to maintain a Union-Find data structure for the nodes. You can import libraries for the Union-Find data structure.
- Prim's algorithm that repeatedly includes an edge with minimum weight to grow the current partial tree. You should run Prim's algorithm with $s = 1$, i.e., start growing the tree from node 1 outwards. Recall that to find the minimum weight edge, you need to maintain a Binary Search Tree (BST) that supports the query of minimum value. You can import libraries for the BST data structure.

Several test cases of input will be provided. For each test case, the main function calls the two algorithms, each of them outputs a MST and its total edge weight.